BITS Pilani WILP AIML M.Tech (Semester 1)

Course: Deep Neural Networks (AIMLCZG511)

HIGH-YIELD MID-SEMESTER EXAM STUDY GUIDE & QUESTION BANK SOLUTIONS

Exam Date: September 19, 2026 (9:00 AM)
Syllabus Coverage: Modules 1 to 6 (Introduction, Perceptron, Linear NN Regression, Linear NN Classification, Deep Feedforward Neural Networks, Convolutional Neural Networks)

PART 1: MASTER FORMULA & CONCEPT CHEAT SHEET

1. The Perceptron (Module 2)

- **Pre-activation (Net Input):**

$$h = \sum_{i=1}^{d} w_i x_i + b = w^T x + b$$

- **Threshold (Step) Activation Function:**

$$\hat{y} = extsign(h) = \begin{cases} 1 & \text{if } h \geq 0 \\ 0 & \text{if } h < 0 \end{cases}$$

- **Perceptron Learning Weight Update Rule (for $t \in \{0, 1\}$):**

$$w_i^{\text{new}} = w_i^{\text{old}} + \eta(t - \hat{y})x_i$$

$$b^{\text{new}} = b^{\text{old}} + \eta(t - \hat{y})$$

  - *Convergence Theorem:* If the training data is **linearly separable**, the Perceptron learning algorithm is guaranteed to converge to a separating hyperplane in a **finite number of steps**.

- *Limitation:* Cannot solve non-linearly separable problems (e.g., XOR) because a single Perceptron can only form a linear decision boundary (hyperplane).

2. Linear NN Regression (Module 3)

- **Model Equation:**

  \hat{y} = w^T x + b$$ (using linear identity activation)

$$J(w) = \frac{1}{2N}\sum_{i=1}^{N}(w^T x^{(i)} - y^{(i)})^2 = \frac{1}{2N}\|Xw - y\|_2^2$$

- **Gradient of Loss (Vector/Matrix Form):**

$$\nabla J(w) = \frac{1}{N}X^T(Xw - y)$$

- **Gradient Descent Update Rule:**

$$w^{(t+1)} = w^{(t)} - \eta \nabla J(w^{(t)}) = w^{(t)} - \frac{\eta}{N}X^T(Xw^{(t)} - y)$$

- **Convergence Monitoring:** If loss increases or becomes `NaN`/`Inf`, **decrease the learning rate** η. If loss oscillates, the learning rate is too large.

3. Linear NN Classification (Module 4)

- **Sigmoid Activation Function (Binary Classification):**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

  - *Derivative:* $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
  - *Range:* $(0, 1)$ - representing probability.
- **Softmax Activation Function (Multi-class Classification, K classes):**

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

  - *Range:* $[0, 1]$ and $\sum_{j=1}^{K} \text{softmax}(z)_j = 1$.
- **Loss Functions:**
  - *Average Binary Cross-Entropy (BCE) Loss:*

$$J(w) = -\frac{1}{N}\sum_{i=1}^{N}\left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right]$$

- *Categorical Cross-Entropy (CCE) Loss (Multi-class):*

$$J(W) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K} y_j^{(i)} \log \hat{y}_j^{(i)}$$

- **Gradients of Cross-Entropy (BCE/CCE) with Sigmoid/Softmax:**

  \nabla \ell_i(w) = (\hat{y}^{(i)} - y^{(i)})x^{(i)}$$ (matches the elegant form of Linear Regression, with $\hat{y}$ as the activation output!)

- **Forward Propagation equations (Layer l):**

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

  (Note: $a^{[0]} = x$ is the input layer)
- **Common Activation Functions & Derivatives:**
  - **Sigmoid:** $g(z) = \frac{1}{1+e^{-z}} \implies g'(z) = g(z)(1-g(z))$ (Max derivative: 0.25 at $z=0$)
  - **Tanh:** $g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \implies g'(z) = 1 - g(z)^2$ (Max derivative: 1.0 at $z=0$)
  - **ReLU:** $g(z) = \max(0, z) \implies g'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$ (Solves vanishing gradient for $z > 0$; prone to "Dying ReLU")
  - **Leaky ReLU:** $g(z) = \max(\alpha z, z) \implies g'(z) = \begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases}$ (Solves dying ReLU)
- **Backpropagation General Equations (Vectorized):**
  Let $\delta^{[l]} = \frac{\partial L}{\partial z^{[l]}}$ represent the error signal at layer l.
  1. **Output Layer Error (L):**

     \delta^{[L]} = a^{[L]} - y$$ (for Cross-Entropy + Sigmoid/Softmax, or MSE + Linear)

$$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot g'^{[l]}(z^{[l]})$$

  2. **Parameter Gradients:**

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]}(a^{[l-1]})^T$$

$$\frac{\partial L}{\partial b^{[l]}} = \delta^{[l]}$$

  3. **Weight Update Rule:**

$$W^{[l]} \leftarrow W^{[l]} - \eta \frac{\partial L}{\partial W^{[l]}}$$

$$b^{[l]} \leftarrow b^{[l]} - \eta \frac{\partial L}{\partial b^{[l]}}$$

5. Convolutional Neural Networks (Module 6)

- **Output Spatial Dimension (O):**

$$O = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

  - Where n is input spatial size (height/width), p is padding, k is kernel/filter size, and s is stride.
- **Trainable Parameter Count in Conv Layer:**

$$\text{Total Parameters} = (k \times k \times C_{\text{in}} + \text{bias}) \times C_{\text{out}}$$

  - Where k is kernel size, C_{in} is input channels, C_{out} is output channels (number of filters), and $\text{bias} = 1$ (usually 1 bias per filter).
  - *Note:* Pooling layers have **ZERO** learnable parameters.
- **Receptive Field (RF_l) at Layer l:**

$$RF_l = RF_{l-1} + (k_l - 1) \times \prod_{i=1}^{l-1} s_i$$

  - Where RF_{l-1} is the receptive field of the previous layer (with $RF_0 = 1$), k_l is the kernel size of layer l, and s_i is the stride of layer i.

PART 2: COMPLETE STEP-BY-STEP QUESTION BANK SOLUTIONS

MODULE 1 & 2: PERCEPTRONS & SINGLE-LAYER NETWORKS

Q1. A bank uses a single-layer neural network to classify loan applications based on income and credit score. During testing, the classifier consistently misclassifies cases where the positive and negative examples cannot be separated by a straight line. Analyze the reason for this behavior and explain what architectural change would make the problem solvable.

- **Analysis of Behavior:** A single-layer neural network (without any hidden layers) is a linear classifier. It can only learn a linear decision boundary (a straight line in 2D space, or

a hyperplane in higher dimensions). When positive and negative examples are non-linearly separable (e.g., in an XOR-like configuration where low-income/high-credit and high-income/low-credit are approved, but other combinations are rejected), no straight line can separate the classes. Thus, a single-layer network is mathematically incapable of achieving 100% accuracy on such datasets.

- **Architectural Change:** The problem can be made solvable by transitioning from a single-layer network to a **Multi-Layer Perceptron (MLP)**. Specifically, we must add **at least one hidden layer with a non-linear activation function** (such as ReLU, Sigmoid, or Tanh). The non-linear activation function in the hidden layer projects the input features into a higher-dimensional space where they become linearly separable, allowing the output layer to draw a valid linear decision boundary.

Q2. A development team increases the number of epochs of a single-layer Perceptron because the model is unable to correctly classify a particular dataset. Explain whether increasing training iterations alone can overcome the limitation of the model.

- **Explanation: No, increasing training iterations (epochs) alone cannot overcome this limitation.**
- According to the **Perceptron Convergence Theorem**, if a dataset is linearly separable, the Perceptron learning algorithm is guaranteed to converge to a separating boundary in a finite number of steps. Conversely, if the dataset is **not** linearly separable, the algorithm will **never** converge, regardless of how many training iterations or epochs are run.
- Instead, the weight vector will continuously oscillate, and the training error will never drop to zero. To solve this, the team must change the model architecture (by adding hidden layers with non-linear activations) rather than simply running more training epochs.

Q3. Consider a neuron with $x = [1, -2]$, $w = [0.6, -0.4]$, and $b = 0.2$. Calculate the net input and output using an appropriate activation function.

- **Step 1: Calculate the net input (z):**

$$z = w^T x + b = (w_1 x_1 + w_2 x_2) + b$$

$$z = (0.6 \times 1) + (-0.4 \times -2) + 0.2$$

$$z = 0.6 + 0.8 + 0.2 = 1.6$$

- **Step 2: Calculate the output (a) using appropriate activation functions:** Since the specific activation function is not restricted, we calculate the output using three standard options:
  1. **Sigmoid Activation** (standard for binary/probability outputs):

$$a = \sigma(z) = \frac{1}{1 + e^{-1.6}} = \frac{1}{1 + 0.2019} \approx 0.8320$$

2. **ReLU Activation** (standard for hidden units):

$$a = \max(0, z) = \max(0, 1.6) = 1.6$$

3. **Tanh Activation** (standard for zero-centered activations):

$$a = \tanh(z) = \frac{e^{1.6} - e^{-1.6}}{e^{1.6} + e^{-1.6}} = \frac{4.9530 - 0.2019}{4.9530 + 0.2019} \approx 0.9217$$

Q4. A classification system initially uses a Perceptron but later requires learning nonlinear decision boundaries. Compare the role of the input, hidden and output layers in the redesigned network.

In the redesigned network (a Multi-Layer Perceptron), the roles of the layers are:

- **Input Layer:** Its sole role is to receive the raw input features (e.g., income, credit score) and pass them directly to the hidden layer. It does not perform any mathematical computation or non-linear transformation.
- **Hidden Layer:** This is the core computational engine for non-linearity. It computes a weighted sum of the inputs, adds a bias, and applies a **non-linear activation function** (e.g., ReLU). This projects the input data into a new, transformed representation space where the classes become linearly separable.
- **Output Layer:** It receives the transformed features from the last hidden layer, computes a weighted sum, and applies a task-specific activation function (e.g., Sigmoid for binary classification, Softmax for multi-class) to generate the final prediction/probabilities. It acts as a linear classifier on top of the non-linear features extracted by the hidden layers.

Q5. A company replaces a shallow network with a deeper network for image recognition. The training accuracy improves significantly, but the engineers are unable to explain why. Explain how depth enables hierarchical feature learning using a suitable example.

- **Hierarchical Feature Learning:** Deep networks learn features in a hierarchical, incremental manner, moving from simple, low-level abstractions to highly complex, high-level abstractions.
- **Example (Face Recognition):**
  - **Early Layers (Layers 1-2):** Learn to detect simple, low-level features such as local edges, lines, gradients, and orientations at specific pixel coordinates.
  - **Mid Layers (Layers 3-4):** Combine the edges and lines detected by early layers to recognize local shapes, textures, and parts of the object (e.g., eyes, nose, ears, mouth, or corners).
  - **Deep Layers (Layers 5+):** Combine the parts to construct global, abstract representations of complete objects (e.g., a complete human face, or individual face categories).

- **Why Depth is Better:** To represent complex functions with a shallow network, one would need an exponentially large number of hidden neurons. A deep network can represent the same complex function with far fewer parameters by composing simpler functions layer by layer, leading to better optimization and generalization.

Q6. A medical-screening model performs well when the classes are linearly separable but fails when several interacting features determine the class. Explain how adding hidden neurons can transform the representation learned by the model.

- **Limitation of Linear Model:** A linear model evaluates each feature independently. If features interact (e.g., XOR relationship where A or B alone does not indicate disease, but their combination does), a linear model fails because it cannot represent the joint multiplicative interaction of features.
- **Transformation by Hidden Neurons:** Adding a hidden layer with non-linear activation functions acts as a **feature intersection detector**.
  - Each hidden neuron learns to fire only for a specific combination of input features (e.g., Hidden Neuron 1 fires if feature x_1 is high AND x_2 is low).
  - This transforms the input space by mapping the non-linear feature interactions into a new coordinate system (representation space) where the target classes are linearly separable.
  - The output layer can then draw a simple linear decision boundary on these transformed features, successfully classifying complex medical cases.

Q7. A team argues that a sufficiently large number of neurons in a single-layer Perceptron should be able to solve any binary classification problem. Evaluate this claim using a suitable decision-boundary example.

- **Evaluation of Claim: The team's claim is false.**
- **Reasoning:** A "single-layer Perceptron" contains only input units and output units, with no hidden layers. Adding more output neurons simply means creating multiple independent linear classifiers. Each output neuron is still strictly limited to learning a linear decision boundary (a straight line in 2D space).
- **Counterexample (XOR Decision Boundary):** Consider the XOR classification problem with four points: $(0, 0)$ and $(1, 1)$ in Class 0; $(0, 1)$ and $(1, 0)$ in Class 1. No straight line can separate these two classes. Adding 100 output neurons to a single-layer Perceptron will just create 100 straight lines, but **none** of them will be able to separate the XOR points. Only by adding a **hidden layer** (creating a Multi-Layer Perceptron) can we solve any non-linear binary classification problem (Universal Approximation Theorem).

MODULE 3 & 4: ACTIVATION FUNCTIONS, LOSSES & DERIVATIVES

Q8. A neural network uses Sigmoid activations in every hidden layer. As the network becomes deeper, the gradients reaching the first hidden layer become extremely small. Explain the phenomenon and recommend a more suitable activation strategy.

- **Phenomenon: Vanishing Gradient Problem:**
  - The derivative of the Sigmoid function is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. The maximum value of this derivative is exactly 0.25 (at $z = 0$), and it rapidly approaches 0 as $|z|$ becomes large (saturation).
  - During backpropagation, the gradient of the loss with respect to the first layer's weights is computed using the chain rule, which involves a product of the Sigmoid derivatives at every subsequent layer:

$$\delta^{[1]} = (W^{[2]})^T \delta^{[2]} \odot \sigma'(z^{[1]}) = \cdots \propto \prod_{l=1}^{L} \sigma'(z^{[l]})$$

  - Multiplying several numbers that are all ≤ 0.25 causes the gradient to decay exponentially with depth ($0.25^L \to 0$). By the time the gradient reaches the first hidden layer, it is virtually zero, meaning early layers learn extremely slowly or not at all.
- **Recommendation:** Replace Sigmoid hidden activations with the **Rectified Linear Unit (ReLU)** activation function:

$$\text{ReLU}(z) = \max(0, z)$$

  Its derivative is exactly 1.0 for all positive inputs ($z > 0$). This constant derivative of 1.0 ensures that gradients flow backward through the network without being scaled down by fractional activation derivatives, effectively preventing the vanishing gradient problem.

Q9. For a neuron with $z = -2$, calculate the output of Sigmoid, ReLU, and Leaky ReLU with $\alpha = 0.01$. Compare their behavior.

- **1. Sigmoid Output:**

$$\sigma(-2) = \frac{1}{1 + e^2} = \frac{1}{1 + 7.3891} \approx 0.1192$$

- **2. ReLU Output:**

$$\text{ReLU}(-2) = \max(0, -2) = 0$$

- **3. Leaky ReLU Output (with $\alpha = 0.01$):**

$$\text{Leaky ReLU}(-2) = \max(0.01 \times -2, -2) = -0.02$$

- **Comparison of Behavior:**

- **Sigmoid** squashes the negative input to a small positive value (0.1192). It has a non-zero output, but its derivative is small, which can slow down training.
- **ReLU** outputs exactly 0. It completely deactivates the neuron, introducing sparsity. However, if a neuron always receives negative inputs, its gradient becomes permanently 0, making it a "dead neuron" that cannot learn.
- **Leaky ReLU** outputs a small negative value (-0.02). It keeps the neuron alive by maintaining a small, active gradient of $\alpha = 0.01$ for negative inputs, preventing the "dying ReLU" problem.

Q10. For $z = 2$, calculate the Sigmoid activation and its derivative. Explain what the derivative indicates about gradient propagation.

- **1. Sigmoid Activation:**

$$a = \sigma(2) = \frac{1}{1 + e^{-2}} = \frac{1}{1 + 0.1353} \approx 0.8808$$

- **2. Derivative of Sigmoid:**

$$\sigma'(2) = \sigma(2)(1 - \sigma(2)) = 0.8808 \times (1 - 0.8808) = 0.8808 \times 0.1192 \approx 0.1050$$

- **3. Indication for Gradient Propagation:** The maximum possible derivative of Sigmoid is 0.25 (at $z = 0$). At $z = 2$, the derivative has already shrunk to 0.1050 (a reduction of 89.5% from 1.0). This indicates that during backpropagation, the incoming gradient will be multiplied by 0.1050 as it passes through this neuron, losing nearly 90% of its magnitude. This demonstrates how even moderate activation values cause rapid decay of gradients in deep networks using Sigmoid activations.

Q11. A deep network uses Tanh activations in several hidden layers. The training process becomes slow even though the loss decreases. Analyze whether Tanh could contribute to the problem.

- **Analysis:** Yes, Tanh can definitely contribute to the slow training process.
- **Reasoning:** The Tanh function is $g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, and its derivative is $g'(z) = 1 - g(z)^2$.
  - Although Tanh is zero-centered (which helps stabilize learning compared to Sigmoid), its derivative is bounded by 1.0 (at $z = 0$) and approaches 0 very quickly as $|z| > 2$ (saturation).
  - If weights are initialized poorly or the learning rate is too high, many activations will fall into the saturated flat regions of Tanh (where $|z|$ is large), causing $g'(z) \to 0$. This leads to the **vanishing gradient problem**, which severely slows down parameter updates and training speed, even if the loss is slowly trickling downwards.

Q12. A computer-vision model replaces Sigmoid hidden units with ReLU. The model trains faster, but some neurons stop responding completely. Identify the possible problem and suggest a modification.

- **Problem Identification: "Dying ReLU" Problem:**
  - The derivative of ReLU is exactly 0 for all negative inputs ($z \leq 0$).
  - If a large gradient update causes the weights of a neuron to become highly negative, the neuron will output 0 for all inputs in the training set ($z < 0$).
  - Since the derivative is 0, the gradient backpropagated through this neuron will be exactly 0. The weights of this neuron will never be updated again, rendering it permanently inactive ("dead").
- **Suggested Modification:** Replace standard ReLU with **Leaky ReLU** (or **Parametric ReLU - PReLU**):

  $$\text{Leaky ReLU}(z) = \max(\alpha z, z) \quad \text{with } \alpha = 0.01$$

  This ensures that for negative inputs, the neuron still has a small non-zero output and a small constant gradient (α), allowing "dead" neurons to receive gradient updates and eventually recover.

Q13. A regression model requires outputs that can take both positive and negative values. Compare the suitability of ReLU, Sigmoid and Tanh for the output layer and justify a choice.

- **Comparison for Output Layer:**
  - **ReLU:** Unsuitable. ReLU outputs ≥ 0. It completely clips all negative predictions to 0, making it impossible to predict negative targets.
  - **Sigmoid:** Unsuitable. Sigmoid squashes outputs strictly to the range $(0, 1)$, which cannot represent negative values or values greater than 1.
  - **Tanh:** Partially suitable but highly limited. Tanh squashes outputs to the range $(-1, 1)$. While it allows negative values, it cannot predict values outside this narrow range and suffers from severe gradient saturation near ± 1.
- **Justified Choice:** The correct choice is **Linear Activation** (also called Identity activation, $a = z$) for the output layer. Linear activation does not squash the output, allowing predictions to take any continuous real value from $-\infty$ to $+\infty$.

Q14. Two hidden-layer designs use ReLU and Leaky ReLU respectively. A developer reports that some units in the ReLU network permanently output zero. Explain why this can happen and how the alternative activation changes the behavior.

- **Why it happens in ReLU:** As explained in Q12, this is the "Dying ReLU" phenomenon. If $z = w^T x + b \leq 0$ for all inputs, the ReLU activation is 0, and its derivative $g'(z)$ is 0.

The update term $\Delta w = -\eta\delta x$ becomes exactly 0 because δ is multiplied by $g'(z) = 0$. The neuron is trapped in a permanent dead state.

- **How Leaky ReLU changes behavior:** Leaky ReLU introduces a small slope α (e.g., 0.01) for negative inputs: $g(z) = \alpha z$ when $z < 0$. Its derivative for negative inputs is $g'(z) = \alpha > 0$. This ensures that even when a neuron is inactive ($z < 0$), a small gradient signal (α) flows back, allowing the optimizer to adjust its weights so that it can eventually fire again for relevant inputs.

MODULE 5: FORWARD & BACKPROPAGATION, LOSSES

Q15. A predictive-maintenance network has two input neurons, two hidden Sigmoid neurons and one Sigmoid output neuron. Given the input, weights and biases, perform complete forward propagation and calculate the final prediction.

Since specific values are not provided, we outline the exact step-by-step mathematical framework of this **2-2-1 Network** and solve a complete concrete numerical example.

- **Network Architecture:**
  - Inputs: $x = [x_1, x_2]^T$
  - Hidden Layer (Layer 1): 2 neurons (h_1, h_2) with Sigmoid activation
  - Output Layer (Layer 2): 1 neuron (o_1) with Sigmoid activation
  - Weights:

$$W^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} b_1^{[2]} \end{bmatrix}$$

- **Step-by-Step Mathematical Steps:**

1. **Compute Hidden Layer pre-activation ($z^{[1]}$):**

$$z_1^{[1]} = w_{11}^{[1]} x_1 + w_{12}^{[1]} x_2 + b_1^{[1]}$$

$$z_2^{[1]} = w_{21}^{[1]} x_1 + w_{22}^{[1]} x_2 + b_2^{[1]}$$

2. **Apply Hidden Layer Sigmoid activation ($a^{[1]}$):**

$$a_1^{[1]} = \sigma(z_1^{[1]}) = \frac{1}{1 + e^{-z_1^{[1]}}}$$

$$a_2^{[1]} = \sigma(z_2^{[1]}) = \frac{1}{1 + e^{-z_2^{[1]}}}$$

3. **Compute Output Layer pre-activation ($z^{[2]}$):**

$$z_1^{[2]} = w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + b_1^{[2]}$$

4. **Compute Output Layer Sigmoid activation ($\hat{y}$):**

$$\hat{y} = a_1^{[2]} = \sigma(z_1^{[2]}) = \frac{1}{1 + e^{-z_1^{[2]}}}$$

- **Concrete Numerical Example (A perfect template for the exam):** Let $x = [0.5, 0.8]^T$. Let the weights and biases be:

$$W^{[1]} = \begin{bmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} 0.5 & 0.6 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} -0.1 \end{bmatrix}$$

  - **Step 1: Compute $z^{[1]}$:**

$$z_1^{[1]} = (0.2 \times 0.5) + (-0.3 \times 0.8) + 0.1 = 0.1 - 0.24 + 0.1 = -0.04$$

$$z_2^{[1]} = (0.4 \times 0.5) + (0.1 \times 0.8) - 0.2 = 0.2 + 0.08 - 0.2 = 0.08$$

  - **Step 2: Compute $a^{[1]}$:**

$$a_1^{[1]} = \sigma(-0.04) = \frac{1}{1 + e^{0.04}} \approx 0.4900$$

$$a_2^{[1]} = \sigma(0.08) = \frac{1}{1 + e^{-0.08}} \approx 0.5200$$

  - **Step 3: Compute $z^{[2]}$:**

$$z_1^{[2]} = (0.5 \times 0.4900) + (0.6 \times 0.5200) - 0.1 = 0.2450 + 0.3120 - 0.1 = 0.4570$$

  - **Step 4: Compute Output $\hat{y}$:**

$$\hat{y} = \sigma(0.4570) = \frac{1}{1 + e^{-0.4570}} \approx 0.6123$$

  - *Final Prediction:* The model predicts a probability of 0.6123.

Q16. For a binary classification network using Binary Cross-Entropy loss, derive the output-layer error term using the chain rule and show why it simplifies to $\delta^{(2)} = \hat{y} - y$.

Let z be the pre-activation of the output layer, $\hat{y} = a^{(2)} = \sigma(z) = \frac{1}{1+e^{-z}}$ be the prediction, and y be the true target binary class. The Binary Cross-Entropy (BCE) loss for a single sample is:

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

We want to find the output-layer error term $\delta^{(2)} = \frac{\partial L}{\partial z}$. By the chain rule:

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

- **Step 1: Compute $\frac{\partial L}{\partial \hat{y}}$:**

$$\frac{\partial L}{\partial \hat{y}} = -\left[\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right] = -\left[\frac{y(1-\hat{y}) - \hat{y}(1-y)}{\hat{y}(1-\hat{y})}\right]$$

$$\frac{\partial L}{\partial \hat{y}} = -\left[\frac{y - y\hat{y} - \hat{y} + y\hat{y}}{\hat{y}(1-\hat{y})}\right] = -\left[\frac{y-\hat{y}}{\hat{y}(1-\hat{y})}\right] = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$$

- **Step 2: Compute $\frac{\partial \hat{y}}{\partial z}$:** Since $\hat{y} = \sigma(z)$, its derivative is:

$$\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \sigma(z)(1-\sigma(z)) = \hat{y}(1-\hat{y})$$

- **Step 3: Combine using the chain rule:**

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \left[\frac{\hat{y}-y}{\hat{y}(1-\hat{y})}\right] \cdot [\hat{y}(1-\hat{y})]$$

  The term $\hat{y}(1-\hat{y})$ cancels out completely:

$$\delta^{(2)} = \hat{y} - y \quad [\mathbf{Q.E.D.}]$$

Q17. A neural network predicts a probability of 0.25 for a sample whose actual class is 1. Calculate the Binary Cross-Entropy loss and interpret the result.

- **1. Loss Calculation:**

$$L = -[y \ln \hat{y} + (1-y)\ln(1-\hat{y})]$$

  Given $y = 1$ and $\hat{y} = 0.25$:

$$L = -[1 \times \ln(0.25) + 0 \times \ln(0.75)] = -\ln(0.25) \approx -(-1.3863) = 1.3863$$

- **2. Interpretation:**
  - The Binary Cross-Entropy loss is 1.3863, which is a high loss value.
  - This is because the network is highly "confident but wrong": it predicted only a 25% probability of the true class being 1, meaning it believed there was a 75% chance of it being class 0.
  - The steep gradient generated by this high loss will drive large weight updates during backpropagation to aggressively correct the network's parameters.

Q18. During backpropagation, the output neuron produces a negative error term. Explain what this indicates about the current prediction and the direction in which the output-layer weights are likely to move.

- **Prediction Indication:**
  - The output-layer error term is $\delta = \hat{y} - y$.
  - If δ is negative ($\delta < 0$), it means $\hat{y} - y < 0 \implies \hat{y} < y$.
  - Since $y \in \{0, 1\}$, this must mean the target is $y = 1$ and the prediction is $\hat{y} < 1$. Thus, the current prediction is an **under-prediction** (underestimating the true target probability).
- **Direction of Output-Layer Weights:**
  - The gradient of the loss with respect to weight w_i is $\frac{\partial L}{\partial w_i} = \delta \cdot a_i$, where $a_i \geq 0$ is the hidden layer activation.
  - The weight update rule is $w_i^{\text{new}} = w_i^{\text{old}} - \eta \frac{\partial L}{\partial w_i} = w_i^{\text{old}} - \eta(\delta a_i)$.
  - Since $\delta < 0$ and $\eta, a_i > 0$, the term $-\eta(\delta a_i)$ becomes positive.
  - Therefore, the output-layer weights w_i associated with active hidden units ($a_i > 0$) will **increase** ($w_i^{\text{new}} > w_i^{\text{old}}$). This increases the net input z in the next forward pass, which in turn increases the prediction $\hat{y}$ toward 1.0.

Q19. A student calculates the output-layer gradient correctly but obtains an incorrect hidden-layer gradient. Identify the role of the chain rule in propagating the error backward and explain where activation derivatives enter the calculation.

- **Role of Chain Rule:**
  - The chain rule allows us to compute the gradient of the loss with respect to early parameters by multiplying local gradients in reverse order: from output back to input.
  - To calculate the error signal at a hidden layer, we must propagate the error backward from the output layer through the weights connecting them:

$$\delta^{[1]} = \frac{\partial L}{\partial z^{[1]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}}$$

- **Where Activation Derivatives Enter:**
  - The activation derivative enters the calculation in the term $\frac{\partial a^{[1]}}{\partial z^{[1]}} = g'^{[1]}(z^{[1]})$.
  - Vectorized, this is:

$$\delta^{[1]} = (W^{[2]})^T \delta^{[2]} \odot g'^{[1]}(z^{[1]})$$

  - The student likely made the mistake of failing to multiply by the element-wise derivative of the hidden activation function ($g'^{[1]}(z^{[1]})$), or calculated the derivative incorrectly. Omitting the activation derivative is a classic error that breaks backpropagation.

Q20. A two-layer classifier gives a confident but incorrect prediction. Explain how the loss and output error signal would respond and why this can produce a comparatively large corrective update.

- **Loss Response:** If the network predicts $\hat{y} \to 0$ when $y = 1$, the BCE loss $L = -\ln(\hat{y})$ approaches ∞. The loss is extremely high.
- **Output Error Signal Response:** The output error signal is $\delta = \hat{y} - y$. Since $\hat{y} \approx 0$ and $y = 1$, we get $\delta = -1.0$ (which is its maximum possible absolute value!).
- **Why this produces a comparatively large corrective update:**
  - The weight gradient is $\frac{\partial L}{\partial w_i} = \delta \cdot a_i$. When $\delta \approx -1.0$, the gradient magnitude is at its absolute peak.
  - Because we use Binary Cross-Entropy loss paired with Sigmoid, the $\sigma'(z)$ term is cancelled out of the error signal.
  - If we had used Mean Squared Error (MSE), the error signal would be $(\hat{y} - y)\sigma'(z)$. For a confident prediction, $|z|$ is large, so $\sigma'(z) \to 0$, which causes the gradient to vanish ("learning slowdown").
  - BCE avoids this completely. The large, unattenuated error signal $\delta \approx -1.0$ ensures a massive corrective weight update, forcing the network to quickly correct its wrong parameters.

MODULE 5: OPTIMIZATION & GRADIENT DESCENT

Q21. An engineer changes the activation function of a hidden layer but keeps the same weights and inputs. Explain which quantities in the forward pass and backpropagation would change and why.

- **In the Forward Pass:**
  - The hidden pre-activations $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ will **remain the same** (since inputs, weights, and biases are unchanged).
  - The hidden activations $a^{[l]} = g(z^{[l]})$ will **change** because the activation function $g(\cdot)$ has changed.
  - Consequently, all subsequent layers' pre-activations ($z^{[l+1]}$) and activations ($a^{[l+1]}$ up to output $\hat{y}$) will **change** because they depend on $a^{[l]}$.
- **In Backpropagation:**
  - The output layer error $\delta^{[L]} = \hat{y} - y$ will **change** (since $\hat{y}$ has changed).
  - The local derivative of the activation function $g'^{[l]}(z^{[l]})$ will **change** (as the function changed).
  - The hidden layer error $\delta^{[l]} = (W^{[l+1]})^T\delta^{[l+1]} \odot g'^{[l]}(z^{[l]})$ will **change** due to both the updated downstream error and the new activation derivative.
  - All weight gradients $\frac{\partial L}{\partial W^{[k]}} = \delta^{[k]}(a^{[k-1]})^T$ will **change** because both the errors $\delta^{[k]}$ and the layer activations $a^{[k-1]}$ have changed.

Q22. A neural-network parameter has $w = 0.8$, learning rate 0.1, and gradient -0.6. Calculate the updated parameter and explain why the parameter increases despite

gradient descent.

- **Calculation:**

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \text{gradient}$$

$$w_{\text{new}} = 0.8 - (0.1 \times -0.6) = 0.8 - (-0.06) = 0.8 + 0.06 = 0.86$$

- **Explanation:**
  - In gradient descent, we update parameters by moving in the direction *opposite* to the gradient to minimize the loss: $w \leftarrow w - \eta \nabla L$.
  - A negative gradient (-0.6) indicates that the loss function is locally sloping downwards to the right (as w increases, loss decreases).
  - Therefore, to decrease the loss, w must be increased. Mathematically, subtracting a negative number is equivalent to adding a positive number, which increases the parameter.

Q23. A training dataset produces gradients 0.4, 0.6, 0.2, and 0.8. Calculate the parameter update using Batch Gradient Descent, SGD using the first sample, and Mini-Batch GD using the first two samples. (Assume learning rate $\eta = 0.1$ and current parameter $w = 0.5$).

Let the gradients of the four samples be $g_1 = 0.4$, $g_2 = 0.6$, $g_3 = 0.2$, and $g_4 = 0.8$.

- **1. Batch Gradient Descent (BGD):** Uses the average of all gradients in the dataset:

$$g_{\text{batch}} = \frac{g_1 + g_2 + g_3 + g_4}{4} = \frac{0.4 + 0.6 + 0.2 + 0.8}{4} = \frac{2.0}{4} = 0.5$$

  Update:

$$w_{\text{new}} = 0.5 - (0.1 \times 0.5) = 0.5 - 0.05 = 0.45$$

- **2. Stochastic Gradient Descent (SGD):** Uses only the gradient of the first sample:

$$g_{\text{sgd}} = g_1 = 0.4$$

  Update:

$$w_{\text{new}} = 0.5 - (0.1 \times 0.4) = 0.5 - 0.04 = 0.46$$

- **3. Mini-Batch Gradient Descent (MBGD):** Uses the average gradient of the first two samples (batch size = 2):

$$g_{\text{mini}} = \frac{g_1 + g_2}{2} = \frac{0.4 + 0.6}{2} = 0.5$$

  Update:

$$w_{\text{new}} = 0.5 - (0.1 \times 0.5) = 0.5 - 0.05 = 0.45$$

Q24. A company trains a DNN using SGD and notices that the loss fluctuates heavily between iterations. Another implementation using Batch GD is extremely slow. Recommend a training strategy and justify your decision.

- **Recommended Strategy: Mini-batch Gradient Descent** (with a moderate batch size, e.g., 32, 64, or 128).
- **Justification:** Mini-batch GD is the industry standard because it combines the advantages of both BGD and SGD while eliminating their flaws:
    1. **Reduces Fluctuations:** By averaging gradients over a batch, it provides a much more stable and accurate estimate of the true gradient than single-sample SGD, significantly smoothing out loss fluctuations.
    2. **High Computational Efficiency (Hardware Acceleration):** Unlike SGD which processes one sample sequentially, mini-batching allows us to leverage GPU vectorization and parallel computing to process multiple samples simultaneously, leading to massive speedups.
    3. **Faster Convergence than BGD:** BGD performs only one update per epoch (after scanning the whole dataset), making it extremely slow for large datasets. Mini-batch GD performs multiple parameter updates per epoch, resulting in much faster overall convergence.

Q25. A model's loss decreases very slowly after several epochs. The developer proposes increasing the learning rate by 20 times. Analyze the possible benefits and risks of this decision.

- **Possible Benefits:**
    - If the model was stuck in a very flat region (saddle point or plateau) where gradients are extremely small, a 20x larger learning rate can accelerate parameter updates, helping the model escape the plateau and speed up training.
- **Possible Risks:**
    - **Divergence:** The optimizer may overshoot the local minimum entirely, climbing up the loss surface and causing the loss to increase exponentially or explode to `NaN`/`Inf`.
    - **Oscillation:** The parameters may oscillate back and forth across the valley of the loss function, never converging to the optimal minimum.
- **Recommendation:** Instead of a static 20x increase, use an **adaptive learning rate scheduler** (e.g., Cosine Annealing, ReduceLROnPlateau) or an adaptive optimizer like **Adam**, which adjusts learning rates dynamically for each parameter based on gradient history.

Q26. Two networks use learning rates of 0.001 and 1.0 respectively. Without performing numerical calculations, explain how their training behavior could differ.

- **Network with $\eta = 0.001$ (Very Small LR):**
  - Training will be extremely **slow, stable, and deterministic**.
  - The loss will decrease smoothly and steadily, but it will require a very large number of epochs to converge.
  - It is at high risk of getting trapped in shallow local minima or plateaus.
- **Network with $\eta = 1.0$ (Very Large LR):**
  - Training will be highly **unstable and erratic**.
  - The loss is likely to bounce around wildly (large oscillations) and may quickly explode or diverge.
  - It is highly likely to overshoot the global minimum and fail to converge altogether.

Q27. A training run reaches a low loss initially but then becomes unstable immediately after the learning rate is increased. Diagnose the likely reason and suggest a more appropriate learning-rate strategy.

- **Diagnosis:** The learning rate was increased beyond the **critical threshold** for the local curvature of the loss surface. The step size became larger than the width of the valley/basin containing the local minimum. This caused the optimizer to overshoot the minimum, climb up the steep walls of the loss valley, and enter a state of numerical instability.
- **Appropriate Learning-Rate Strategy:**
  1. **Learning Rate Decay / Scheduling:** Start with a relatively high learning rate to learn fast, and then gradually *decrease* (rather than increase) the learning rate as training progresses (e.g., using step decay, exponential decay, or cosine annealing) to fine-tune weights near the minimum.
  2. **Adaptive Optimizers (Adam/RMSprop):** These algorithms maintain separate, running adaptive learning rates for each parameter, automatically scaling steps down in steep regions and scaling them up in flat plateaus.

Q28. Two teams use identical networks and datasets but obtain different convergence behavior because one uses full-batch updates and the other uses mini-batches. Explain how the update frequency and gradient estimates can account for the difference.

- **1. Update Frequency:**
  - *Full-Batch:* Performs only **one parameter update per epoch** (after calculating gradients over the entire dataset). For a dataset of size $N = 100,000$, it updates the weights once per epoch. Learning is slow in terms of calendar time.
  - *Mini-Batch:* Performs N/B **updates per epoch** (where B is batch size). If $B = 100$, it updates the weights $1,000$ times in a single epoch, converging much faster.

- **2. Gradient Estimates:**
  - *Full-Batch:* Computes the **exact, deterministic gradient** of the entire dataset. The optimization path is smooth, direct, and stable, but it can easily get trapped in the nearest local minimum or saddle point.
  - *Mini-Batch:* Computes a **noisy, stochastic estimate** of the gradient. This noise acts as a regularizer, providing helpful "random kicks" that allow the optimizer to jump out of shallow local minima and saddle points, often leading to a better final generalization.

Q29. A 10-layer network receives an output gradient of 0.8. The gradient multiplier at every layer is 0.25. Calculate the approximate gradient reaching the first layer.

- **Calculation:** Let the output gradient at Layer 10 be $\delta^{[10]} = 0.8$. The gradient propagates backward from Layer 10 to Layer 1, passing through 9 layer-to-layer transitions. At each transition, it is multiplied by the gradient multiplier of 0.25.

$$\delta^{[1]} = \delta^{[10]} \times (0.25)^9$$

$$\delta^{[1]} = 0.8 \times \left(\frac{1}{4}\right)^9 = 0.8 \times \frac{1}{262144}$$

$$\delta^{[1]} = 0.8 \times 3.8147 \times 10^{-6} \approx 3.0518 \times 10^{-6}$$

- **Implication:** The gradient reaching the first layer is extremely small (≈ 0.000003), demonstrating a severe **vanishing gradient problem**. The first layer will learn virtually nothing.

Q30. Repeat the previous analysis when the gradient multiplier is 1.8. Identify the training problem.

- **Calculation:**

$$\delta^{[1]} = \delta^{[10]} \times (1.8)^9$$

$$\delta^{[1]} = 0.8 \times 198.3592 \approx 158.6874$$

- **Identification of Training Problem: Exploding Gradient Problem:**
  - Since the multiplier is greater than 1 (1.8), the gradient grows exponentially as it backpropagates.
  - By the time it reaches the first layer, its magnitude has increased from 0.8 to ≈ 158.7 (a 198-fold increase).
  - This causes massive weight updates in early layers, destroying learned patterns, making training highly unstable, and eventually leading to overflow errors where activations and weights become `NaN` or `Inf`.

MODULE 5: UNSTABLE GRADIENTS, INITIALIZATION & TRAINING

Q31. An engineer observes that the final layers of a DNN learn rapidly while the first few layers hardly change. Diagnose the likely gradient problem and explain why it occurs.

- **Diagnosis: Vanishing Gradient Problem.**
- **Explanation of why it occurs:**
  - Gradients are computed during backpropagation from the output layer backward toward the input layer.
  - At each layer, the backpropagated error is multiplied by the derivative of that layer's activation function and the layer's weight matrix:

$$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot g'^{[l]}(z^{[l]})$$

  - If the network uses activations with derivatives less than 1 (like Sigmoid where $g'(z) \leq 0.25$ or Tanh where $g'(z) \leq 1.0$), or if the weight matrices have small spectral norms, the gradient is scaled down at each step.
  - Since the final layers are close to the loss function, they receive the full, strong, unattenuated gradient signal and learn rapidly. However, as the gradient travels backward through many layers, it decays exponentially, reaching the first few layers with near-zero magnitude. Consequently, early layers hardly change.

Q32. A deep network suddenly produces extremely large parameter updates and eventually generates NaN values during training. Identify the likely cause and suggest two remedies.

- **Likely Cause: Exploding Gradient Problem:**
  - If the product of weight matrices and activation derivatives is greater than 1 (due to poor initialization or large weights), gradients grow exponentially as they flow backward.
  - This leads to massive parameter updates, making the weights excessively large until they exceed the floating-point limits of the system, resulting in `NaN` (Not a Number) or `Inf` values.
- **Two Suggested Remedies:**
  1. **Gradient Clipping:** Intercept and clip gradients during backpropagation if their norm or absolute value exceeds a specified threshold (e.g., clipping gradients to have a max norm of 1.0).
  2. **He / Xavier Initialization:** Properly initialize weights with a variance scaled inversely by the number of input/output units, which mathematically prevents gradients from growing or decaying exponentially at the start of training.

Q33. A research group is choosing between Xavier initialization, He initialization, Batch Normalization, Residual Connections and Gradient Clipping. For a network suffering from unstable gradients, select two techniques and explain why they are appropriate.

- **Selection:** We select **Batch Normalization (BatchNorm)** and **Residual Connections (Skip Connections)**.
- **Explanation of Suitability:**
  1. **Batch Normalization:**
     - *Why appropriate:* BatchNorm normalizes the inputs of each layer across the mini-batch, ensuring they maintain zero mean and unit variance.
     - *Gradient Flow impact:* This keeps layer activations from entering saturated regions (where activation derivatives are near zero), thus maintaining large, stable gradients throughout training. It also reduces sensitivity to weight initialization.
  2. **Residual Connections:**
     - *Why appropriate:* Residual connections add shortcut paths that bypass one or more layers: $a^{[l]} = g(z^{[l]}) + a^{[l-2]}$.
     - *Gradient Flow impact:* During backpropagation, these shortcuts act as **gradient superhighways**, allowing the gradient to flow directly through the identity addition operator without any attenuation. This mathematically guarantees that a strong gradient signal reaches the earliest layers, completely eliminating the vanishing gradient problem.

Q34. A network using many Sigmoid layers shows almost no change in its early-layer weights. Explain how repeated multiplication of derivatives can create this behavior.

- **Explanation:**
  - By the chain rule, the gradient reaching Layer 1 in a network of L layers is:

$$\delta^{[1]} = \delta^{[L]} \times W^{[L]}\sigma'(z^{[L-1]}) \times W^{[L-1]}\sigma'(z^{[L-2]}) \times \cdots \times W^{[2]}\sigma'(z^{[1]})$$

  - The derivative of the Sigmoid function is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
  - The mathematical upper bound of the Sigmoid derivative is 0.25 (which occurs only when $z = 0$, and is even smaller for other inputs).
  - In a deep network with many layers, the term $\prod_{l=1}^{L-1} \sigma'(z^{[l]})$ represents repeated multiplication of numbers that are all ≤ 0.25.
  - For example, in a 10-layer network, this product scales down the gradient by at least $0.25^9 \approx 3.8 \times 10^{-6}$. This exponential decay shrinks the gradient to near-zero, leaving early-layer weights completely unchanged.

Q35. A deep network's gradients become very large only for a small number of training batches. Explain why gradient clipping could be useful and distinguish its purpose from that of careful weight initialization.

- **Why Gradient Clipping is Useful:**
  - In complex loss landscapes, the optimizer may occasionally encounter "cliffs" or extremely steep regions. When a mini-batch enters these regions, the computed gradient suddenly spikes to a massive value, causing the model to take a giant step that destroys hours of learned parameters.
  - Gradient clipping acts as a dynamic runtime safety net: if a gradient exceeds a threshold, it is scaled down to a safe magnitude, allowing the model to navigate steep cliffs safely without exploding.
- **Distinction from Careful Weight Initialization:**
  - **Careful Weight Initialization (e.g., He, Xavier):** Is a **static, initialization-phase** technique. It sets the starting weights of the network to ensure stable activations and gradients at the *beginning* of training. However, it cannot guarantee that gradients won't become unstable or explode *later* in training as weights evolve.
  - **Gradient Clipping:** Is a **dynamic, runtime-phase** safeguard. It actively monitors and corrects gradient magnitudes at *every single training step* during the backward pass, protecting the network throughout the entire optimization lifecycle.

MODULE 6: CONVOLUTIONAL NEURAL NETWORKS (CNNs)

Q36. A CNN receives a $28 \times 28 \times 1$ image. It contains a 3×3 convolution with 8 filters, stride 1 and no padding, followed by 2×2 max pooling with stride 2. Calculate the output dimensions after both layers.

- **Step 1: Calculate after the Convolutional Layer:**
  - Input size: $n = 28$, Kernel size: $k = 3$, Stride: $s = 1$, Padding: $p = 0$ (no padding), Filters: $C_{\text{out}} = 8$.
  - Output spatial dimension (O_{conv}):

$$O_{\text{conv}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 = \left\lfloor \frac{28 + 0 - 3}{1} \right\rfloor + 1 = 25 + 1 = 26$$

  - Output dimension after Conv: $26 \times 26 \times 8$.
- **Step 2: Calculate after the Max Pooling Layer:**
  - Input size: $n_{\text{pool}} = 26$, Pool size: $k_{\text{pool}} = 2$, Stride: $s_{\text{pool}} = 2$, Padding: $p = 0$.
  - Output spatial dimension (O_{pool}):

$$O_{\text{pool}} = \left\lfloor \frac{n_{\text{pool}} + 2p - k_{\text{pool}}}{s_{\text{pool}}} \right\rfloor + 1 = \left\lfloor \frac{26 + 0 - 2}{2} \right\rfloor + 1 = \frac{24}{2} + 1 = 12 + 1 = 13$$

  - Channels remain unchanged at 8.

- **Final Output Dimension:** $13 \times 13 \times 8$.

Q37. A CNN receives an RGB image of size $64 \times 64 \times 3$. A convolution layer contains 16 filters of size 3×3, stride 1 and padding 1. Calculate its output size and number of trainable parameters.

- **1. Calculate Output Size:**
  - Input size: $n = 64$, Kernel size: $k = 3$, Stride: $s = 1$, Padding: $p = 1$, Filters: $C_{\text{out}} = 16$.
  - Output spatial dimension (O):

$$O = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 = \left\lfloor \frac{64 + 2(1) - 3}{1} \right\rfloor + 1 = 63 + 1 = 64$$

  - Output Size: $64 \times 64 \times 16$ (spatial dimensions are preserved due to "same" padding).

- **2. Calculate Trainable Parameters:**
  - Input channels $C_{\text{in}} = 3$ (RGB), Kernel size $k = 3$, Output filters $C_{\text{out}} = 16$.
  - Parameter Formula:

$$\text{Parameters} = (k \times k \times C_{\text{in}} + \text{bias}) \times C_{\text{out}}$$

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 28 \times 16 = 448$$

  - There are 448 **trainable parameters** (432 weights and 16 biases).

Q38. A CNN has a convolution layer with 32 filters of size 5×5, receiving 16 input channels. Calculate the number of trainable parameters including biases.

- **Calculation:**
  - Kernel size $k = 5$
  - Input channels $C_{\text{in}} = 16$
  - Output filters $C_{\text{out}} = 32$
  - Parameter Formula:

$$\text{Total Parameters} = (k \times k \times C_{\text{in}} + 1) \times C_{\text{out}}$$

$$\text{Total Parameters} = (5 \times 5 \times 16 + 1) \times 32$$

$$\text{Total Parameters} = (25 \times 16 + 1) \times 32$$

$$\text{Total Parameters} = (400 + 1) \times 32 = 401 \times 32 = 12,832$$

- **Total Parameters:** $12,832$ **trainable parameters** (12,800 weights and 32 biases).

Q39. A student claims that pooling layers increase the number of trainable parameters because they introduce additional operations. Evaluate the claim.

- **Evaluation: The student's claim is completely incorrect.**
- **Explanation:**
  - Pooling layers (such as max pooling or average pooling) perform fixed, pre-defined mathematical operations over a sliding local window (e.g., selecting the maximum value or computing the average value).
  - They do **not** have any weights or biases. Therefore, they introduce **zero (0) trainable parameters** into the network.
  - Although they introduce floating-point operations (computational cost), their primary function is to perform downsampling (reducing spatial dimensions), which actually **drastically reduces** the number of activations and subsequent trainable parameters in downstream fully-connected layers.

Q40. A surveillance company considers two models: one that directly flattens a $128 \times 128 \times 3$ image and another that first applies convolution and pooling. Explain why the second approach is generally more suitable for image analysis.

The second approach (Convolution and Pooling) is far superior due to three core properties:

1. **Preservation of Spatial Locality:** Image pixels are highly correlated with their neighbors. Flattening directly into a fully-connected layer destroys all spatial relationships (treating pixels on opposite corners of the image with the same weight structure as adjacent pixels). Convolutional layers preserve local spatial context.
2. **Parameter Efficiency & Prevention of Overfitting:**
   - Directly flattening a $128 \times 128 \times 3$ image results in $49,152$ inputs. A single fully-connected layer with 1,000 hidden units would require $49,152 \times 1,000 \approx 49 \text{ million}$ parameters, which will cause severe overfitting.
   - Convolutional layers share weights across the image, drastically reducing the parameter count (e.g., a $3 \times 3 \times 3$ filter has only 28 parameters), allowing the network to generalize far better on small datasets.
3. **Translation Invariance:** A convolutional filter scans the entire image, enabling it to detect a feature (e.g., a face) regardless of where it appears in the frame. Directly flattened networks are highly sensitive to the exact pixel coordinates of an object and fail if it shifts.

Q41. A CNN receives a $32 \times 32 \times 3$ image and applies a 3×3 convolution with stride 2 and padding 1. Determine the output dimensions and explain how changing the stride affects spatial resolution.

- **1. Determine Output Spatial Dimensions:**

- Input size: $n = 32$, Kernel: $k = 3$, Stride: $s = 2$, Padding: $p = 1$, Filters: C_{out} (unspecified, we'll call it C_{out}).
- Output spatial dimension (O):

$$O = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 = \left\lfloor \frac{32 + 2(1) - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{31}{2} \right\rfloor + 1 = 15 + 1 = 16$$

- The output spatial dimension is $16 \times 16 \times C_{\text{out}}$.

- **2. Effect of Stride on Spatial Resolution:**
  - The stride determines the step size of the filter as it slides across the input.
  - **Increasing the stride** (e.g., from 1 to 2) performs downsampling, which **reduces the output spatial resolution** by skipping input locations. This reduces computational cost (FLOPs) and memory footprint in subsequent layers but risks losing fine-grained, high-frequency spatial details.
  - **Decreasing the stride** (e.g., to 1) preserves high spatial resolution and fine structures but increases computational complexity.

Q42. A model designer doubles the number of filters in a convolutional layer while keeping the kernel size unchanged. Explain which dimensions of the parameter count and feature representation are affected.

- **Impact on Parameter Count:**
  - The number of parameters in a convolutional layer is $(k \times k \times C_{\text{in}} + 1) \times C_{\text{out}}$.
  - Since C_{out} (the number of filters) is doubled, the parameter count of **this layer will double** (a linear, 100% increase).
  - Furthermore, since the output of this layer becomes the input to the next convolutional layer, the parameter count of the **next layer** will also double (because its input channels C_{in} have doubled).
- **Impact on Feature Representation:**
  - The output channel depth is doubled. This allows the network to learn and extract **twice as many unique, distinct feature maps** (feature representations) simultaneously from the input (e.g., detecting more diverse textures, edges, or parts). It significantly increases the representational capacity of the model.

Q43. A CNN contains a sequence of 3×3 convolutions and 2×2 pooling layers. Determine the receptive field of a neuron in a later feature map with respect to the original image. Clearly show the calculation layer by layer.

To demonstrate clearly, we use a standard, representative VGG-style sequential architecture:

- **Layer 1:** Convolution ($k_1 = 3$, Stride $s_1 = 1$)
- **Layer 2:** Max Pooling ($k_2 = 2$, Stride $s_2 = 2$)

- **Layer 3:** Convolution ($k_3 = 3$, Stride $s_3 = 1$)

We use the layer-by-layer Receptive Field (RF) formula:

$$RF_l = RF_{l-1} + (k_l - 1) \times J_{l-1}$$

where $J_{l-1} = \prod_{i=1}^{l-1} s_i$ is the cumulative stride of all preceding layers, and we initialize $RF_0 = 1$ and $J_0 = 1$.

- **Layer 1:** 3×3 **Conv with stride** 1 **(**$k_1 = 3, s_1 = 1$**):**
  - Previous: $RF_0 = 1, J_0 = 1$.
  - Calculation:

$$RF_1 = RF_0 + (k_1 - 1) \times J_0 = 1 + (3 - 1) \times 1 = 3$$

  - Current cumulative stride: $J_1 = J_0 \times s_1 = 1 \times 1 = 1$.
  - *Receptive Field after Layer 1:* 3×3.
- **Layer 2:** 2×2 **Max Pooling with stride** 2 **(**$k_2 = 2, s_2 = 2$**):**
  - Previous: $RF_1 = 3, J_1 = 1$.
  - Calculation:

$$RF_2 = RF_1 + (k_2 - 1) \times J_1 = 3 + (2 - 1) \times 1 = 4$$

  - Current cumulative stride: $J_2 = J_1 \times s_2 = 1 \times 2 = 2$.
  - *Receptive Field after Layer 2:* 4×4.
- **Layer 3:** 3×3 **Conv with stride** 1 **(**$k_3 = 3, s_3 = 1$**):**
  - Previous: $RF_2 = 4, J_2 = 2$.
  - Calculation:

$$RF_3 = RF_2 + (k_3 - 1) \times J_2 = 4 + (3 - 1) \times 2 = 4 + 2 \times 2 = 8$$

  - *Receptive Field after Layer 3:* 8×8.

Q44. A CNN designer replaces one 7×7 convolution with three consecutive 3×3 convolutions. Analyze the effect on receptive field, parameters and number of nonlinear transformations.

- **1. Effect on Receptive Field:**
  - Receptive field of one 7×7 convolution with stride 1 is 7.
  - Receptive field of three consecutive 3×3 convolutions with stride 1:
    - Layer 1: $RF_1 = 3$
    - Layer 2: $RF_2 = RF_1 + (3 - 1) \times 1 = 5$

        - Layer 3: $RF_3 = RF_2 + (3-1) \times 1 = 7$
    - **Result:** The receptive field is **exactly the same (7×7)**. The three 3×3 layers cover the same input context.

- **2. Effect on Parameter Count:**
    - Let the input and output channels be C (constant depth).
    - For one 7×7 convolution layer:

$$\text{Parameters}_{7\times7} = (7 \times 7 \times C) \times C = 49C^2$$

    - For three 3×3 convolution layers:

$$\text{Parameters}_{3\times3} = 3 \times [(3 \times 3 \times C) \times C] = 27C^2$$

    - **Result:** Replacing 7×7 with three 3×3 convolutions achieves a 45% **parameter reduction** ($27/49 \approx 0.55$). This significantly lowers the risk of overfitting and decreases memory usage.

- **3. Effect on Nonlinear Transformations:**
    - A single 7×7 layer has only **one activation function** (e.g., ReLU) applied after it.
    - Three consecutive 3×3 layers have **three activation functions** applied (one after each conv layer).
    - **Result:** This introduces more non-linearities into the network, allowing it to learn much more complex, discriminative, and richer feature representations.

Q45. A company chooses between LeNet-5 and VGG-style architecture for a complex image-classification problem. Explain why the deeper architecture may learn richer representations.

- **VGG-style (Deeper)** is much more suitable than **LeNet-5 (Shallower)** for complex tasks because:
    1. **Hierarchical Depth:** A deeper architecture contains more layers, allowing it to construct a highly hierarchical representation of features (from simple edges in layer 1 to complex parts in layer 8, to abstract object categories in layer 16). LeNet-5 is too shallow to construct deep semantic abstractions.
    2. **More Non-Linearities:** VGG stacks multiple 3×3 convolutions back-to-back, each followed by a non-linear activation (ReLU). This introduces many more non-linear decision boundaries compared to LeNet-5, allowing it to model highly complex, non-linear class manifolds.
    3. **Parameter Efficiency:** Stacking multiple 3×3 convolutions instead of using larger filters (LeNet-5 uses 5×5) allows VGG to achieve a larger receptive field with fewer parameters, preventing overfitting on complex datasets.

Q46. A VGG-style CNN receives a $224 \times 224 \times 3$ image and applies three 2×2 max-pooling operations with stride 2. Determine the spatial dimensions before the fully connected layer.

The input spatial size is 224×224. Each 2×2 max-pooling operation with stride 2 halves the spatial dimensions.

- **After Pooling 1:**

$$O_1 = \frac{224}{2} = 112 \implies 112 \times 112$$

- **After Pooling 2:**

$$O_2 = \frac{112}{2} = 56 \implies 56 \times 56$$

- **After Pooling 3:**

$$O_3 = \frac{56}{2} = 28 \implies 28 \times 28$$

- **Final Spatial Dimensions:** The spatial dimensions before the fully connected layer are 28×28 (width $\times$ height). The overall volume will be $28 \times 28 \times C_{\text{last}}$, where C_{last} is the channel depth of the final convolutional layer.

Q47. An autonomous-vehicle vision system requires detection of increasingly complex features—from edges to textures to object parts. Explain how a CNN's hierarchical feature extraction enables this progression. Compare this with directly connecting the pixels to a fully connected layer.

- **Hierarchical Progression in CNNs:**
  - Early convolutional layers have small receptive fields and learn to detect low-level, local features such as edges, curves, and high-contrast lines.
  - As we go deeper, the feature maps of early layers are combined through subsequent convolutions and downsampled through pooling. This increases the receptive field.
  - Middle layers combine local edges into textures and object parts (e.g., wheels, headlights, or windshields of a car).
  - Deep layers combine these parts to form high-level semantic representations of complete objects (e.g., pedestrian, vehicle, or stop sign).
- **Comparison with Directly Connected FC Layer:**
  - **FC Layer on raw pixels:** Discards all spatial structure, treats every pixel independently, and requires an enormous number of parameters (leading to catastrophic overfitting). It cannot learn translation-invariant feature hierarchies and fails if an object shifts by even a few pixels.

    - **CNN:** Captures local spatial dependencies via local receptive fields, reduces parameters through weight sharing, and naturally builds a translation-invariant feature hierarchy.

Q48. A CNN designer observes that increasing the stride of an early convolution reduces the size of later feature maps. Explain the trade-off between computational efficiency, spatial detail and receptive field.

Increasing the stride of an early convolution layer has three major effects:

1. **Computational Efficiency: Increases significantly.** A larger stride downsamples the feature map early, which quadratically reduces the height and width of all subsequent layers' feature maps. This drastically lowers the number of floating-point operations (FLOPs) and GPU memory footprint.
2. **Spatial Detail: Decreases significantly.** Larger strides skip pixel alignments, acting as a coarse downsampling mechanism. This discards high-frequency, fine-grained spatial details, which can make it very difficult to detect or locate small objects/boundaries later.
3. **Receptive Field: Increases much faster.** A larger stride scales up the cumulative stride multiplier J_{l-1} of the network, which causes the receptive fields of all subsequent layers to grow much more rapidly with depth.

- **Trade-off Summary:** A larger stride yields high computational speed and fast receptive field growth at the cost of losing fine-grained spatial detail and localization accuracy.

Q49. An image-classification model performs well on large objects but poorly on small defects. Analyze how pooling, stride and receptive-field growth could contribute to this behavior and suggest an architectural adjustment.

- **Analysis of Failure on Small Defects:**
    - **Pooling and Large Strides:** These operations are designed to discard high-frequency spatial noise to make the model translation-invariant. However, they perform severe downsampling. For a small defect (e.g., a 2-pixel scratch), a sequence of pooling and strides will literally "pool away" and completely erase the defect's activations in early layers, making it invisible to deeper layers.
    - **Rapid Receptive-Field Growth:** Deep layers have massive receptive fields that focus on the global context of the object. They smooth out tiny local anomalies, making the network blind to small, localized defects.
- **Suggested Architectural Adjustments:**
    1. **Reduce/Eliminate Early Pooling:** Reduce the number of pooling layers or strides in the early layers to preserve high spatial resolution for as long as possible.
    2. **Dilated (Atrous) Convolutions:** Use dilated convolutions instead of pooling. They expand the receptive field without downsampling or losing spatial resolution.

3. **Skip Connections (U-Net or FPN-style):** Add skip connections (like in Feature Pyramid Networks) that copy and concatenate high-resolution, un-pooled features from early layers directly to the deep classification layers. This ensures that fine-grained local details are preserved and combined with global semantic context.

YOU ARE FULLY PREPARED FOR THE EXAM! REVISE THESE PAGES AND ACES THE MID-SEM! GOOD LUCK!